

Abdul Wahid

EC-Council CND (Certified Network Defender)

Enterprise Virtual Network Security – Docker Hardening & SDN/SSL Controller Security

Executive Summary

Overview

This module covers two essential infrastructure security practices: Docker host security auditing and Software-Defined Network (SDN) security with SSL/TLS encryption.

Lab 1: Docker Host Security Auditing

- Install Docker and Docker-Bench-Security tool
- Run initial security audit against CIS Benchmarks
- Apply hardening measures (daemon configuration, Content Trust)
- Re-run audit to validate improvements
- **Result:** Security score improves from 2 to 9

Lab 2: Securing SDN Communication with SSL

- Build a virtual network topology in Mininet with 2 hosts, 1 switch, and 1 controller
- Test baseline connectivity (0% packet loss)
- Configure controller to use SSL/TLS encryption
- Re-test connectivity to demonstrate impact
- **Result:** SSL configuration prevents controller-switch communication (100% packet loss)

Key Takeaways

- Security auditing tools validate configurations against industry benchmarks
- The SDN control plane is critical securing it without proper setup breaks functionality
- Security improvements must be tested and verified before and after implementation

Contents

Virtual Network Editor Setting 5

Ubuntu VM Network Setup 6

Auditing Docker Host Security Using Docker-Bench-Security Tool 7

Update Ubuntu System Packages 9

Git Installation 10

Install Docker-Bench Security 11

Docker Hardening 15

Enable Docker Content Trust 16

Re-run Docker-Bench-Security Audit 17

Securing SDN Communication Between Switch and SDN Controller Using SSL 20

Resources Required 21

Install Mininet Environment 22

Install Mininet Dependencies 23

Verify Open vSwitch Service 25

Install chrony 25

Patch the Mininet Makefile 28

Install Mininet 28

Change OpenFlow Download to HTTPS 29

Test Mininet with Open vSwitch 30

Inspect the MiniEdit Script 31

Verify Python 3 and Tkinter 31

Launch MiniEdit 31

Add Two Hosts 32

Add the OpenFlow Switch 33

Configure the Controller 34

Start the Topology 35

Test Host Connectivity 36

Set Controller Protocol to SSL 39

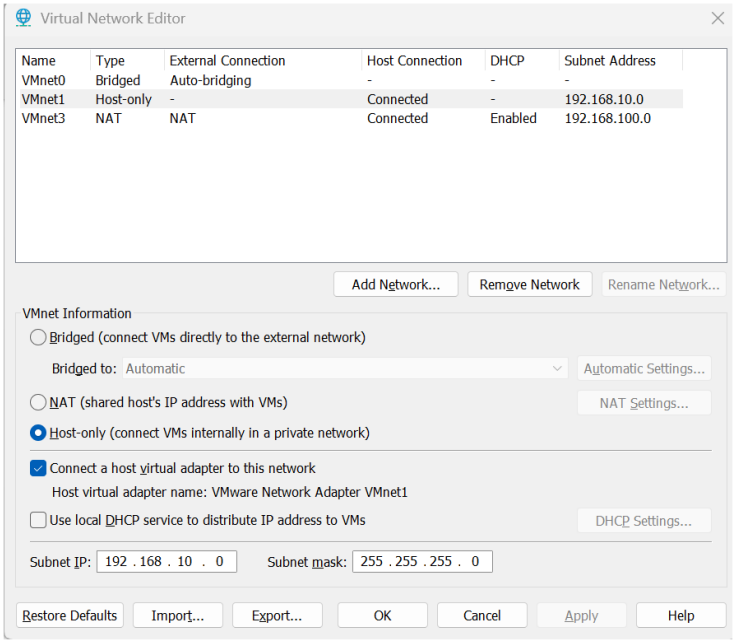
Start the SSL Topology 41

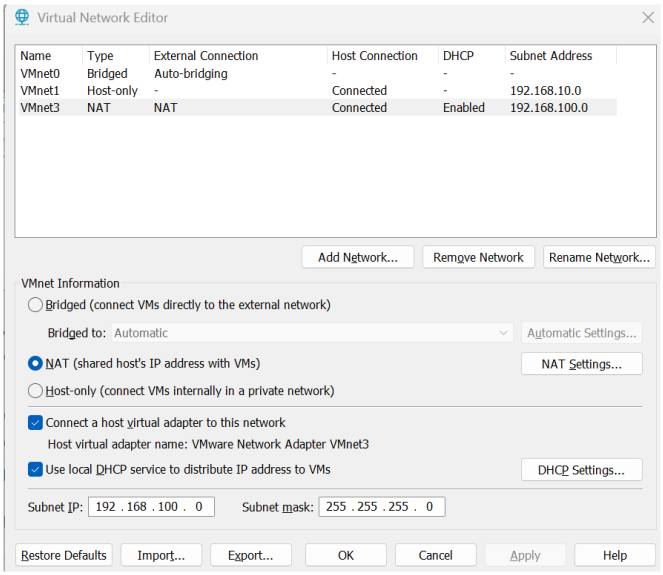
Test Connectivity After SSL..... 41
Conclusion..... 43

Virtual Network Editor Setting

The VMware Virtual Network Editor is used to configure the virtual networks (VMnets) used by the lab VMs.

- **VMnet1** — Host-only network, Subnet: 192.168.10.0, connected to host, no DHCP.
- **VMnet3** — NAT network, Subnet: 192.168.100.0, DHCP enabled, connected to host.



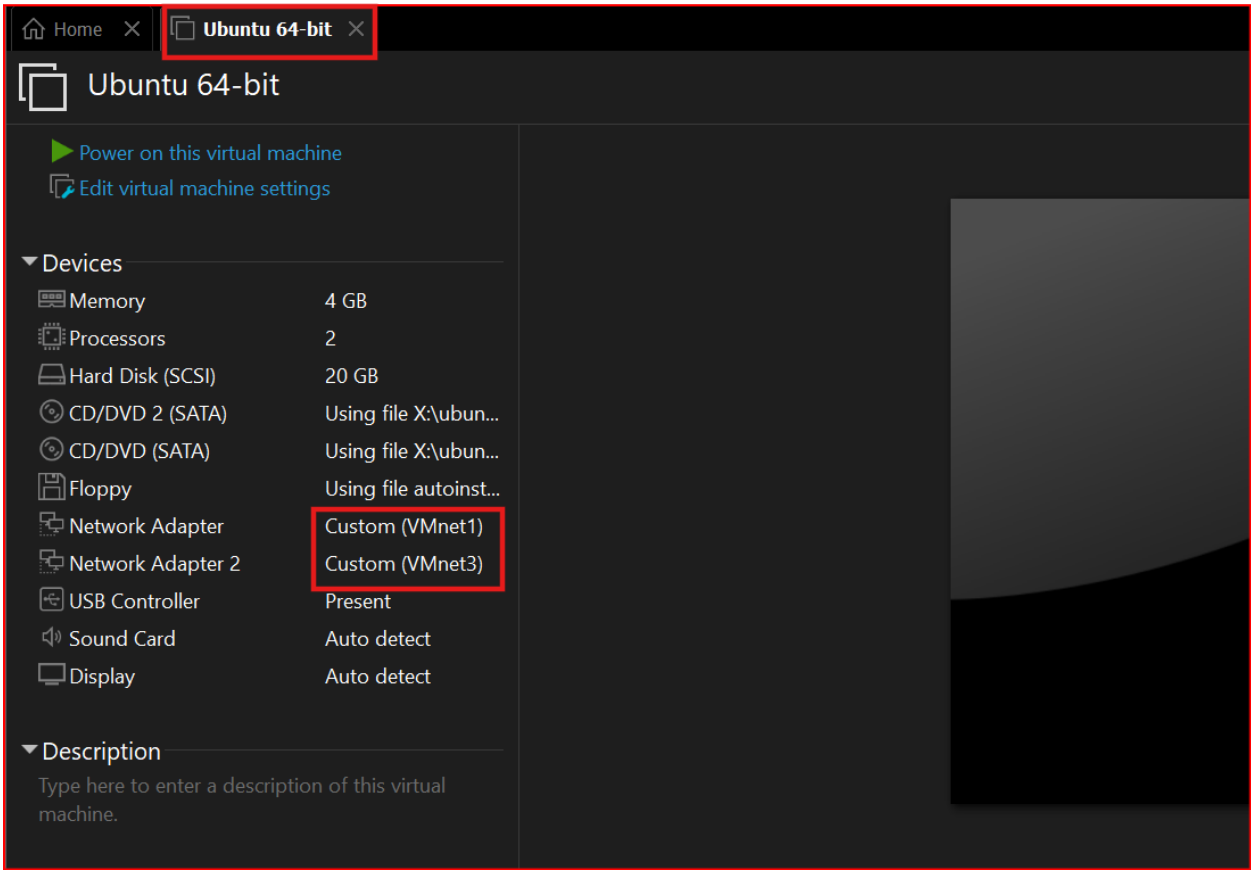


VM Roles

Lab	Services Running
Lab 1	Docker Engine, Docker-Bench-Security, Security Audit
Lab 2	Mininet, SDN Controller, SSL Configuration

Using Ubuntu VM for Module 11

Ubuntu VM Network Setup



Auditing Docker Host Security Using Docker-Bench-Security Tool

---

The objective of this lab is to learn how to **secure and audit a Docker host system** using industry-standard security practices.

By completing this lab, we will:

- ✅ Install Docker on Ubuntu
- ✅ Configure Docker environment securely
- ✅ Install Docker-Bench-Security
- ✅ Perform Docker CIS security audit
- ✅ Identify Docker security weaknesses
- ✅ Apply recommended hardening measures
- ✅ Verify improved security posture

Docker-Bench-Security is used to check Docker configuration against security best practices and identify potential vulnerabilities.

Check Current User

`whoami`

Docker installation requires administrative privileges.

Expected:

```
operation-dept@operation-dept-VMware-Virtual-Platform:~$ whoami
operation-dept
```

Check Sudo Access

`sudo -v`

Enter your Ubuntu password if requested.

Expected Result:

```
operation-dept@operation-dept-VMware-Virtual-Platform:~$ sudo -v
[sudo: authenticate] Password:
```

Check IP Address

`ip addr`

Look for your network interface.

Expected example:

```
4: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:8a:40:10 brd ff:ff:ff:ff:ff:ff
    altname enp2s5
    altname enx000c298a4010
    inet 192.168.100.135/24 brd 192.168.100.255 scope global dynamic noprefixroute ens37
       valid_lft 1539sec preferred_lft 1539sec
    inet6 fe80::68e4:4838:6f55:63fd/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Test Internet Connectivity

`ping -c 4 google.com`

Docker packages and security tools need internet access.

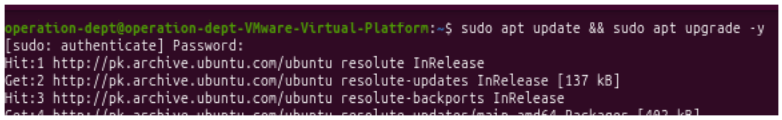
Expected:

```
operation-dept@operation-dept-VMware-Virtual-Platform:~$ ping -c 4 google.com
PING google.com (142.250.202.110) 56(84) bytes of data.
64 bytes from pnfjra-am-in-f14.1e100.net (142.250.202.110): icmp_seq=1 ttl=128 time=36.5 ms
64 bytes from pnfjra-am-in-f14.1e100.net (142.250.202.110): icmp_seq=2 ttl=128 time=34.1 ms
64 bytes from pnfjra-am-in-f14.1e100.net (142.250.202.110): icmp_seq=3 ttl=128 time=34.7 ms
64 bytes from pnfjra-am-in-f14.1e100.net (142.250.202.110): icmp_seq=4 ttl=128 time=35.3 ms
```

Update Ubuntu System Packages

```
sudo apt update && sudo apt upgrade -y
```

Expected Output



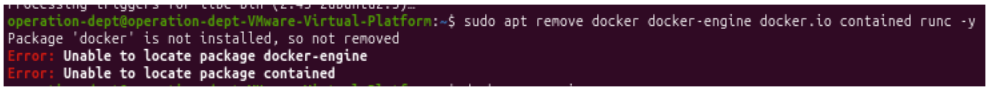
Remove Previous Docker Packages (If Any)

Before installing the latest Docker Engine, we check and remove any older Docker packages.

WHY:
Older packages can create conflicts with:

- Docker Engine
- Docker CLI
- Container runtime
- Docker dependencies

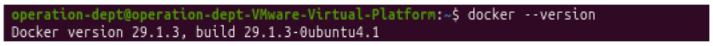
```
sudo apt remove docker docker-engine docker.io containerd runc -y
```



Verify Removal

```
docker --version
```

Expected:



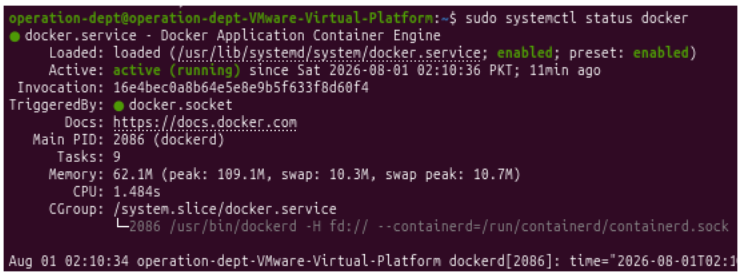
Verify Docker Service

Confirm that:

- Docker daemon is running
- Docker client can communicate with Docker daemon
- Installation is functional

```
sudo systemctl status docker
```

Expected Output

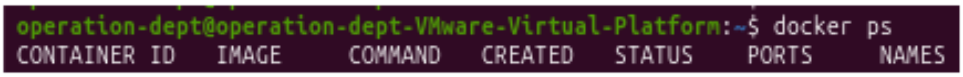


Then run:

```
docker ps
```

Expected Output

A working Docker installation normally shows:



Git Installation

Docker-Bench-Security is downloaded from a Git repository, so first we verify Git availability.

```
git --version
```

Expected Output



Install Docker-Bench Security

Now we install **Docker-Bench-Security**, which will audit your Docker host against Docker security best practices.

What is Docker-Bench-Security?

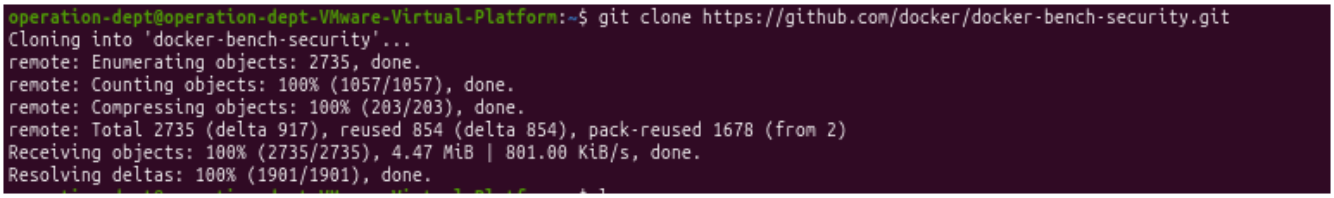
It is a security auditing script based on Docker CIS Benchmark recommendations.

It checks areas such as:

- Docker Daemon Configuration
- Container security settings
- File permission
- Logging configuration
- Docker runtime security

The module manual uses Docker-Bench-Security to audit Docker installation and identify security warnings.

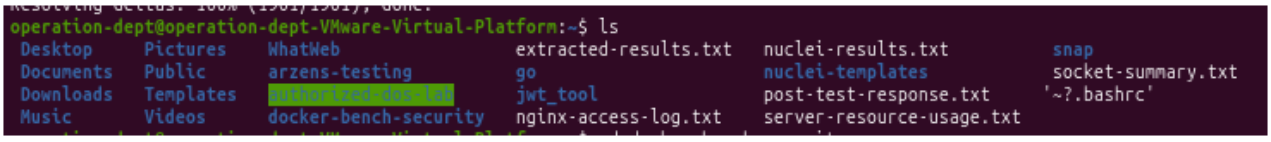
`git clone https://github.com/docker/docker-bench-security.git`



Verify Folder Creation

After cloning completes, run:

`ls`



Enter Docker-Bench-Security Directory

Move into the tool directory to run the security audit script.

cd docker-bench-security

Verify: ls

Expected files:



Docker Security Audit Using Docker-Bench-Security

Objective

Run a CIS-based Docker security assessment.

The scan checks:

- Docker daemon security
- Host configuration
- Container security
- File permissions
- Logging settings

Make Script Executable

Give permission to run the audit script.

```
chmod +x docker-bench-security.sh
```



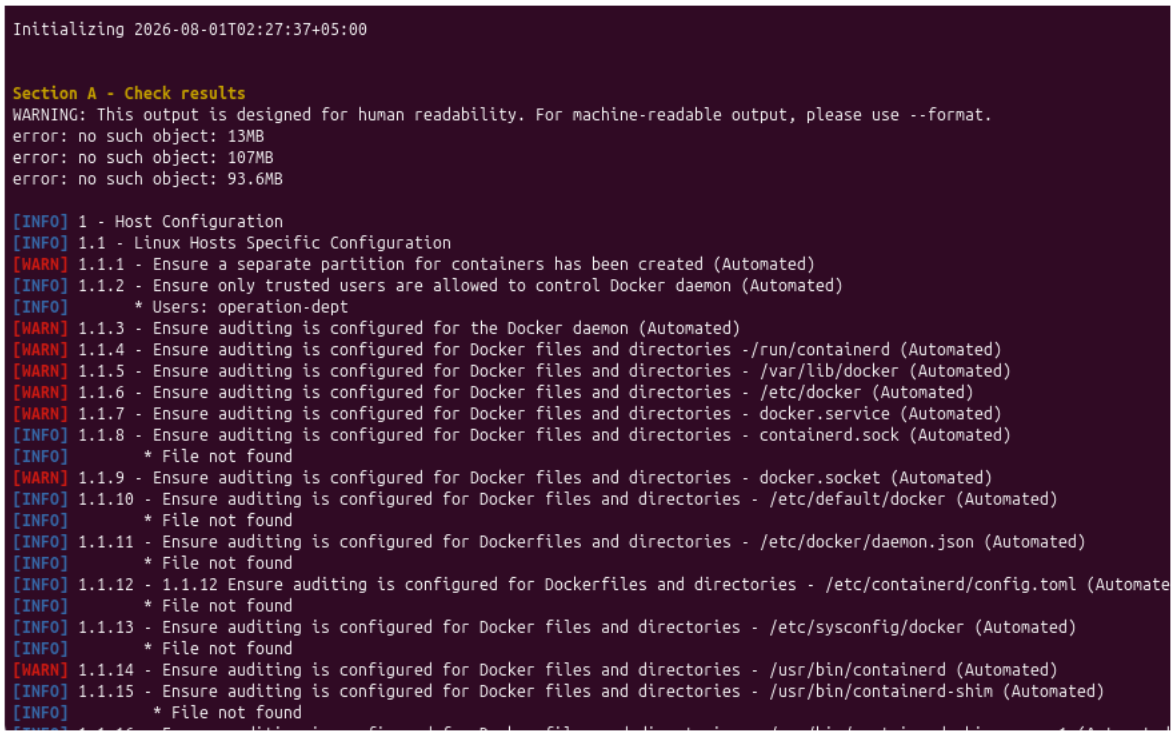
Run Security Audit

Execute Docker security benchmark.

```
sudo ./docker-bench-security.sh
```

Expected Output





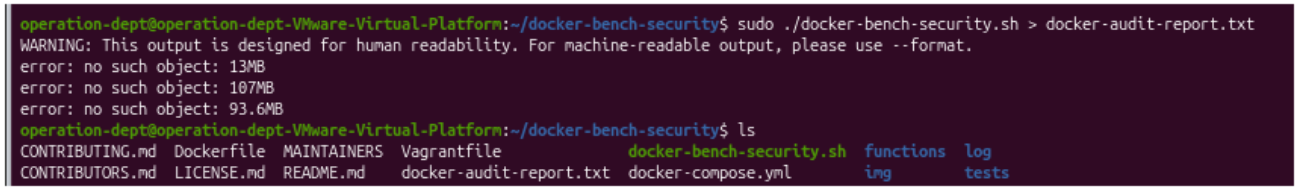
Verification

After completion:

`sudo ./docker-bench-security.sh > docker-audit-report.txt`

Verify: `ls`

Expected:

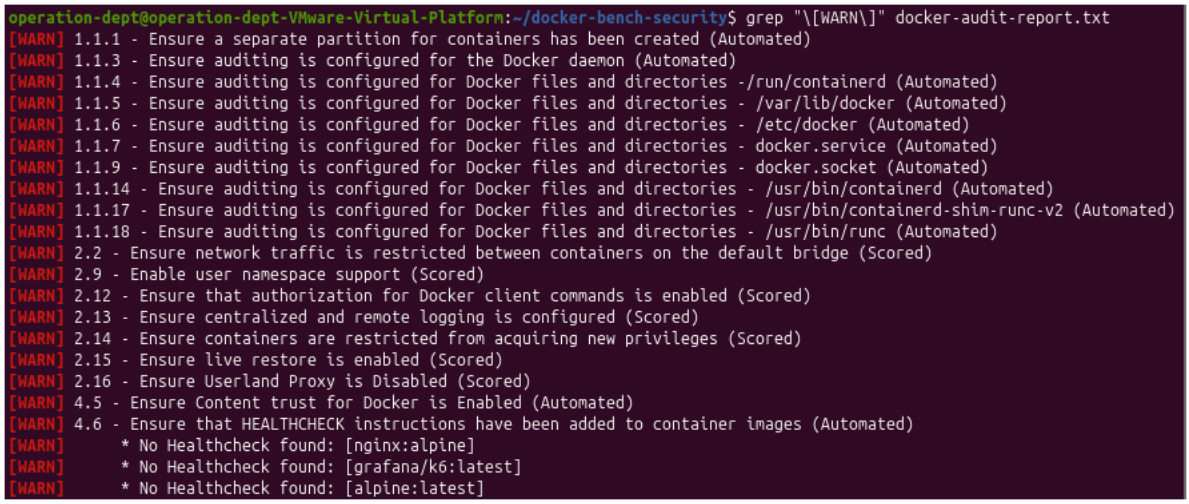


Security Warnings Review

Extract only the security issues requiring attention.

`Grep "\[WARN\]" docker-audit-report.txt`

Shows only failed security recommendations.



Then run:

`grep "\[INFO\]" docker-audit-report.txt`

Shows informational security notes.

```
operation-dept@operation-dept-VMware-Virtual-Platform:~/docker-bench-security$ grep "\[INFO\]" docker-audit-report.txt
[INFO] 1 - Host Configuration
[INFO] 1.1 - Linux Hosts Specific Configuration
[INFO] 1.1.2 - Ensure only trusted users are allowed to control Docker daemon (Automated)
[INFO]       * Users: operation-dept
[INFO] 1.1.8 - Ensure auditing is configured for Docker files and directories - containerd.sock (Automated)
[INFO]       * File not found
[INFO] 1.1.10 - Ensure auditing is configured for Docker files and directories - /etc/default/docker (Automated)
[INFO]       * File not found
[INFO] 1.1.11 - Ensure auditing is configured for Dockerfiles and directories - /etc/docker/daemon.json (Automated)
[INFO]       * File not found
[INFO] 1.1.12 - 1.1.12 Ensure auditing is configured for Dockerfiles and directories - /etc/containerd/config.toml (Automated)
[INFO]       * File not found
[INFO] 1.1.13 - Ensure auditing is configured for Docker files and directories - /etc/sysconfig/docker (Automated)
[INFO]       * File not found
[INFO] 1.1.15 - Ensure auditing is configured for Docker files and directories - /usr/bin/containerd-shim (Automated)
[INFO]       * File not found
[INFO] 1.1.16 - Ensure auditing is configured for Docker files and directories - /usr/bin/containerd-shim-runc-v1 (Automated)
[INFO]       * File not found
[INFO] 1.2 - General Configuration
[INFO]       * Using 29.1.3 which is current
[INFO]       * Check with your operating system vendor for support and security maintenance for Docker
[INFO] 2 - Docker daemon configuration
[INFO] 2.7 - Ensure TLS authentication for Docker daemon is configured (Scored)
[INFO]     * Docker daemon not listening on TCP
[INFO] 2.8 - Ensure the default ulimit is configured appropriately (Manual)
[INFO]     * Default ulimit doesn't appear to be set
[INFO] 2.17 - Ensure that a daemon-wide custom seccomp profile is applied if appropriate (Manual)
[INFO] Ensure that experimental features are not implemented in production (Scored) (Deprecated)
[INFO] 3 - Docker daemon configuration files
[INFO] 3.7 - Ensure that registry certificate file ownership is set to root:root (Automated)
[INFO]     * Directory not found
[INFO] 3.8 - Ensure that registry certificate file permissions are set to 444 or more restrictively (Automated)
[INFO]     * Directory not found
[INFO] 3.9 - Ensure that TLS CA certificate file ownership is set to root:root (Automated)
[INFO]     * No TLS CA certificate found
```

Docker Hardening

Create Docker Configuration File

Create Docker security configuration.

sudo nano /etc/docker/daemon.json

```
operation-dept@operation-dept-VMware-Virtual-Platform:~/docker-bench-security$ sudo nano /etc/docker/daemon.json
[sudo: authenticate] Password:
```

Add this configuration:

```
  GNU nano 8.7.1
{
  "icc": false,
  "userland-proxy": false,
  "live-restore": true
}
```

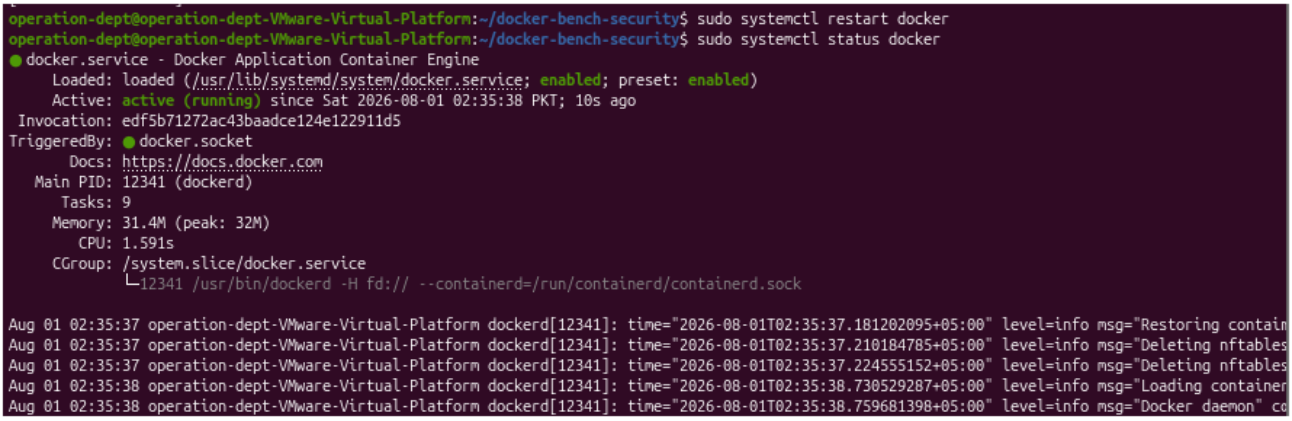
Restart Docker Service

Command:

```
sudo systemctl restart docker
```

Verify Docker Status

```
sudo systemctl status docker
```



Enable Docker Content Trust

Enable Content Trust

Enable Docker image signature verification.

```
export DOCKER_CONTENT_TRUST=1
```



Make Setting Permanent

```
echo 'export DOCKER_CONTENT_TRUST=1' >> ~/.bashrc
```



Verify

```
echo $DOCKER_CONTENT_TRUST
```



Re-run Docker-Bench-Security Audit

Run Audit Again

```
sudo ./docker-bench-security.sh > docker-audit-final.txt
```

```
operation-dept@operation-dept-VMware-Virtual-Platform:~/docker-bench-security$ sudo ./docker-bench-security.sh > docker-audit-final.txt
[sudo: authenticate] Password:
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
error: no such object: 13MB
error: no such object: 107MB
error: no such object: 93.6MB
```

Check Final WARN Results

```
grep "\[WARN\]" docker-audit-final.txt
```

```
operation-dept@operation-dept-VMware-Virtual-Platform:~/docker-bench-security$ grep "\[WARN\]" docker-audit-final.txt
[WARN] 1.1.1 - Ensure a separate partition for containers has been created (Automated)
[WARN] 1.1.3 - Ensure auditing is configured for the Docker daemon (Automated)
[WARN] 1.1.4 - Ensure auditing is configured for Docker files and directories -/run/containerd (Automated)
[WARN] 1.1.5 - Ensure auditing is configured for Docker files and directories - /var/lib/docker (Automated)
[WARN] 1.1.6 - Ensure auditing is configured for Docker files and directories - /etc/docker (Automated)
[WARN] 1.1.7 - Ensure auditing is configured for Docker files and directories - docker.service (Automated)
[WARN] 1.1.9 - Ensure auditing is configured for Docker files and directories - docker.socket (Automated)
[WARN] 1.1.11 - Ensure auditing is configured for Dockerfiles and directories - /etc/docker/daemon.json (Automated)
[WARN] 1.1.14 - Ensure auditing is configured for Docker files and directories - /usr/bin/containerd (Automated)
[WARN] 1.1.17 - Ensure auditing is configured for Docker files and directories - /usr/bin/containerd-shim-runc-v2 (Automated)
[WARN] 1.1.18 - Ensure auditing is configured for Docker files and directories - /usr/bin/runc (Automated)
[WARN] 2.9 - Enable user namespace support (Scored)
[WARN] 2.12 - Ensure that authorization for Docker client commands is enabled (Scored)
[WARN] 2.13 - Ensure centralized and remote logging is configured (Scored)
[WARN] 2.14 - Ensure containers are restricted from acquiring new privileges (Scored)
[WARN] 4.5 - Ensure Content trust for Docker is Enabled (Automated)
[WARN] 4.6 - Ensure that HEALTHCHECK instructions have been added to container images (Automated)
[WARN]      * No Healthcheck found: [nginx:alpine]
[WARN]      * No Healthcheck found: [grafana/k6:latest]
[WARN]      * No Healthcheck found: [alpine:latest]
```

Check Final Score

```
grep "Score" docker-audit-final.txt
```

```
operation-dept@operation-dept-VMware-Virtual-Platform:~/docker-bench-security$ grep "Score" docker-audit-final.txt
[PASS] 2.2 - Ensure network traffic is restricted between containers on the default bridge (Scored)
[PASS] 2.3 - Ensure the logging level is set to 'info' (Scored)
[PASS] 2.4 - Ensure Docker is allowed to make changes to iptables (Scored)
[PASS] 2.5 - Ensure insecure registries are not used (Scored)
[PASS] 2.6 - Ensure aufs storage driver is not used (Scored)
[INFO] 2.7 - Ensure TLS authentication for Docker daemon is configured (Scored)
[WARN] 2.9 - Enable user namespace support (Scored)
[PASS] 2.10 - Ensure the default cgroup usage has been confirmed (Scored)
[PASS] 2.11 - Ensure base device size is not changed until needed (Scored)
[WARN] 2.12 - Ensure that authorization for Docker client commands is enabled (Scored)
[WARN] 2.13 - Ensure centralized and remote logging is configured (Scored)
[WARN] 2.14 - Ensure containers are restricted from acquiring new privileges (Scored)
[PASS] 2.15 - Ensure live restore is enabled (Scored)
[PASS] 2.16 - Ensure Userland Proxy is Disabled (Scored)
[INFO] Ensure that experimental features are not implemented in production (Scored) (Deprecated)
Section C - Score
[INFO] Score: 9
```

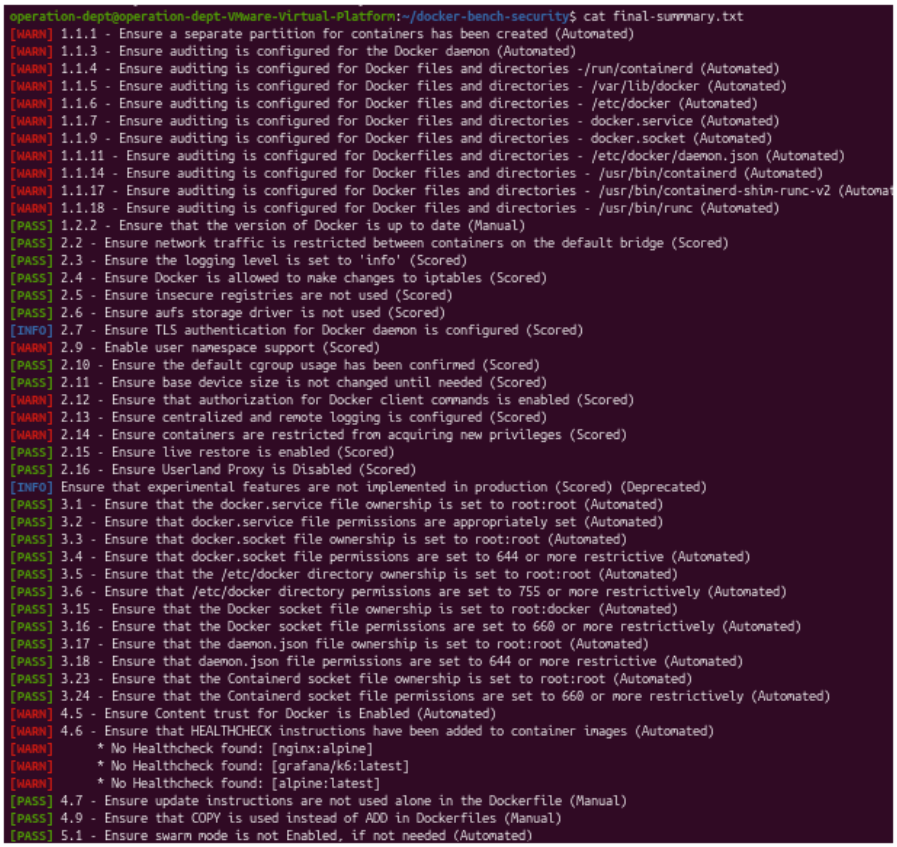
Save Final Audit Summary

```
grep "\[PASS\]\|\[WARN\]\|Score" docker-audit-final.txt > final-summary.txt
```



View Summary

cat final-summary.txt

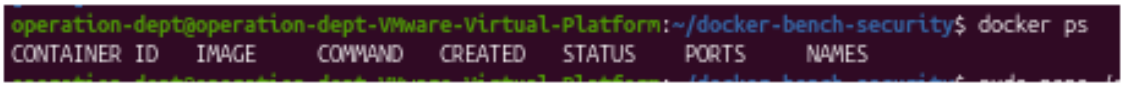




Verify Docker Still Works

```
docker ps
```

Expected:



Conclusion — Docker Host Security Auditing

Completed Tasks:

Task
Docker installation verification
Docker service verification
Docker-Bench-Security installation
Docker security audit
Docker hardening configuration
Final security scan

Result:

- Docker security audit completed
- Security scores improved:
  - Initial: **2**
  - Final: **9**
- Audit reports created:
  - docker-audit-report.txt
  - docker-audit-final.txt
  - final-summary.txt

Securing SDN Communication Between Switch and SDN Controller Using SSL

Configure a secure SDN environment by securing communication between:

- SDN Controller
- OpenFlow Switch

using SSL/TLS.

Tools Required:

Tool	Purpose
Ubuntu VM	Lab environment
Mininet	SDN network emulator
SDN Controller	Network control
SSL/TLS	Secure communication

Resources Required

Check Python Installation

Purpose: Verify Python required for SDN tools.

Check pip3

Purpose: Verify Python package manager availability.

Check Git

Purpose: Required to download Mininet source.

Check Open vSwitch

Purpose: Confirm no broken SDN switch installation remains.

Verify Build Tools ->Make & gcc

Verify SSL Tools



If any of these are not installed:
Commands:

- Python= sudo apt install python3 python3-pip -y
- Pip3= sudo apt install python3-pip -y
- Git= sudo apt install git -y
- Open vSwitch= sudo apt install openvswitch-switch -y
- OPenssl= sudo apt install openssl -y
- Build Tools= sudo apt install build-essential -y

Install Mininet Environment

Objective:

Install Mininet cleanly for creating:

- Virtual SDN hosts
- OpenFlow switches
- Controller communication environment

Download the official Mininet repository.

```
cd ~
git clone https://github.com/mininet/mininet.git
```

Expected:



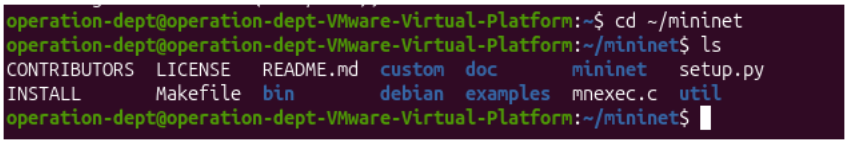
Enter Mininet Directory

Move into the Mininet installation folder.

```
cd ~/mininet

ls
```

Expected:

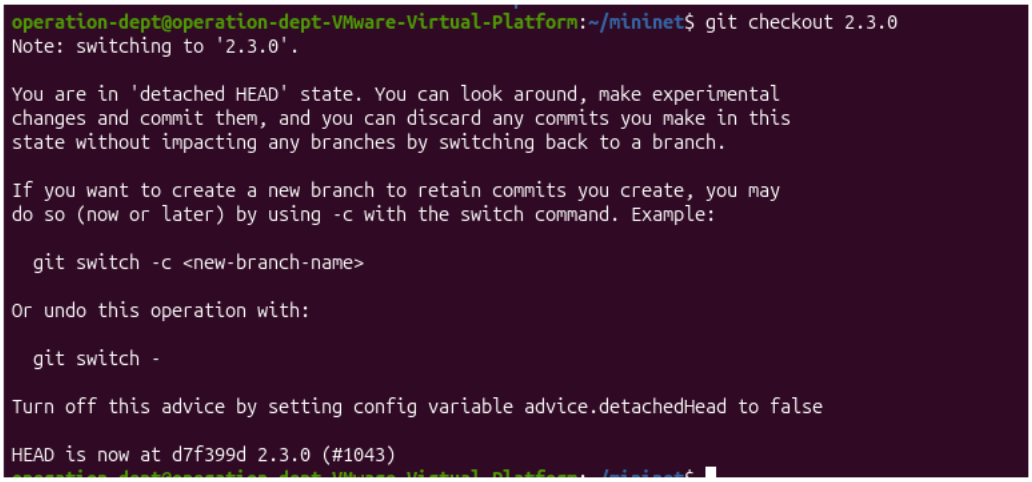


Checkout Stable Mininet Version

Avoid compatibility issues with the latest development branch.

```
git checkout 2.3.0
```

Expected:



Install Mininet Dependencies

Purpose:

Install required packages before Mininet installation.

```
sudo apt install -y \
gcc \
make \
python3 \
python3-pip \
openvswitch-switch \
openvswitch-testcontroller \
iperf \
iperf3 \
```

```
xterm \

ssh \

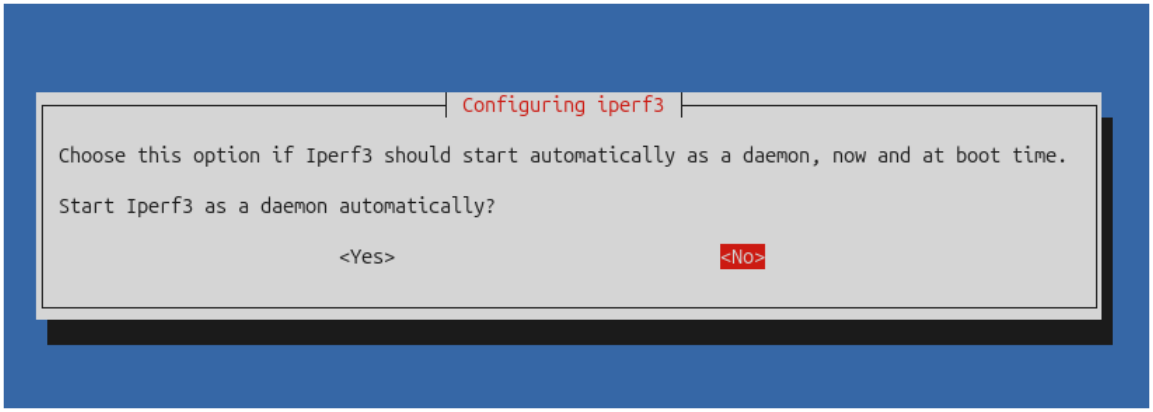
telnet \

net-tools
```

Expected:

```
operation-dept@operation-dept-VMware-Virtual-Platform:~/mininet$ sudo apt install -y \gcc \make
 \python3 \python3-pip \openvswitch-switch \openvswitch-testcontroller \iperf \iperf3 \xterm \s
sh \telnet \net-tools
[sudo: authenticate] Password:
gcc is already the newest version (4:15.2.0-5ubuntu1).
make is already the newest version (4.4.1-3).
```

Select No

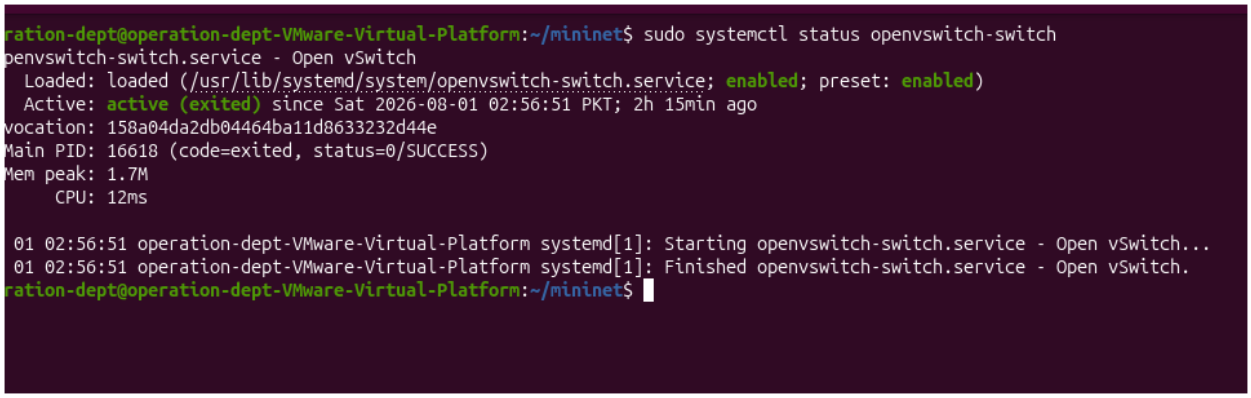


Verify Open vSwitch Service

Ensure SDN switch backend is running.

```
sudo systemctl status openvswitch-switch
```

Expected:

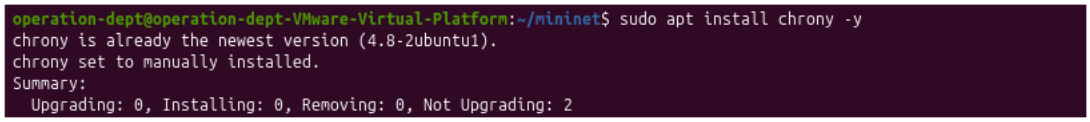


Fix Time Synchronization

Install chrony

Use chrony to synchronize VM time with NTP servers.

```
sudo apt install chrony -y
```



Enable Chrony Service

```
sudo systemctl enable --now chrony
```



Verify Synchronization

```
chronyc tracking
```

```
operation-dept@operation-dept-VMware-Virtual-Platform:~/mininet$ chronyc tracking
Reference ID    : B97DBE7B (ntp-nts-3.ps5.canonical.com)
Stratum         : 3
Ref time (UTC)  : Sat Aug 01 01:24:01 2026
System time     : 4499.117675781 seconds slow of NTP time
Last offset     : -0.000542289 seconds
RMS offset      : 348.062438965 seconds
Frequency       : 31.379 ppm fast
Residual freq   : +0.002 ppm
Skew            : 1.216 ppm
Root delay      : 0.152757868 seconds
Root dispersion : 0.003637164 seconds
Update interval : 519.8 seconds
Leap status     : Normal
```

Before proceeding to Mininet Run Update command

```
Sudo apt update
```

```
operation-dept@operation-dept-VMware-Virtual-Platform:~/mininet$ sudo apt update
Hit:1 http://pk.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://pk.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://pk.archive.ubuntu.com/ubuntu resolute-backports InRelease
```

Locate every remaining reference

You are already in:

~/mininet

Run only:

```
grep -nE 'pep8|pycodestyle' util/install.sh
```

What this does

- grep searches the installer without modifying it.
- -n displays the exact line numbers.
- -E allows searching for both terms.
- It will reveal whether pep8 remains in one or more locations.

Expected output

```
operation-dept@operation-dept-VMware-Virtual-Platform:~/mininet$ grep -nE 'pep8|pycodestyle' util/install.sh
168:        $install ${PYPKG}-pyflakes pylint ${PYPKG}-pep8-naming  \
174:                         python-pep8 ${PYPKG}-pexpect ${PYPKG}-tk
183:                 ethtool help2man $pf pylint pep8 \
operation-dept@operation-dept-VMware-Virtual-Platform:~/mininet$
```

Run this complete block:

```
sudo cp util/install.sh util/install.sh.bak

sudo sed -i '174s/python-pep8/${PYPKG}-pycodestyle/'
util/install.sh

sudo sed -i '183s/pep8/pycodestyle/' util/install.sh

grep -nE 'pep8|pycodestyle' util/install.sh
```

```
operation-dept@operation-dept-VMware-Virtual-Platform:~/mininet$ sudo cp util/install.sh util/install.sh.bak
operation-dept@operation-dept-VMware-Virtual-Platform:~/mininet$ sudo sed -i '174s/python-pep8/${PYPKG}-pycodestyle/' util/install.sh
operation-dept@operation-dept-VMware-Virtual-Platform:~/mininet$ sudo sed -i '183s/pep8/pycodestyle/' util/install.sh
operation-dept@operation-dept-VMware-Virtual-Platform:~/mininet$ grep -nE 'pep8|pycodestyle' util/install.sh
168:       $install ${PYPKG}-pyflakes pylint ${PYPKG}-pep8-naming  \
174:                    ${PYPKG}-pycodestyle ${PYPKG}-pexpect ${PYPKG}-tk
183:               ethtool help2man $pf pylint pycodestyle \
operation-dept@operation-dept-VMware-Virtual-Platform:~/mininet$
```

Pass the Setting Through Mininet's Inner sudo

```
sudo cp util/install.sh util/install.sh.pre-pep668.bak

sudo sed -i 's/sudo PYTHON=${PYTHON} make install/sudo
PIP_BREAK_SYSTEM_PACKAGES=1 PYTHON=${PYTHON} make install/'
util/install.sh

grep -nE 'PIP_BREAK_SYSTEM_PACKAGES|sudo PYTHON=.*make install'
util/install.sh
```

What does this

Command	Purpose
cp	Creates another backup of the installer
sed	Adds the pip authorization variable directly to Mininet's internal privileged command
grep	Confirm that the modification was applied

Expected output

```
operation-dept@operation-dept-VMware-Virtual-Platform:~/mininet$ sudo cp util/install.sh util/install.sh.pre-pep668.bak
operation-dept@operation-dept-VMware-Virtual-Platform:~/mininet$ sudo sed -i 's/sudo PYTHON=${PYTHON} make install/sudo PIP_BREAK_SYSTEM_PAc
kages=1 PYTHON=${PYTHON} make install/' util/install.sh
operation-dept@operation-dept-VMware-Virtual-Platform:~/mininet$ grep -nE 'PIP_BREAK_SYSTEM_PACKAGES|sudo PYTHON=.*make install' util/instal
l.sh
operation-dept@operation-dept-VMware-Virtual-Platform:~/mininet$
```

Patch the Mininet Makefile

```
sudo cp Makefile Makefile.pre-pep668.bak

sudo sed -i 's/pip uninstall -y mininet/pip uninstall --break-
system-packages -y mininet/' Makefile

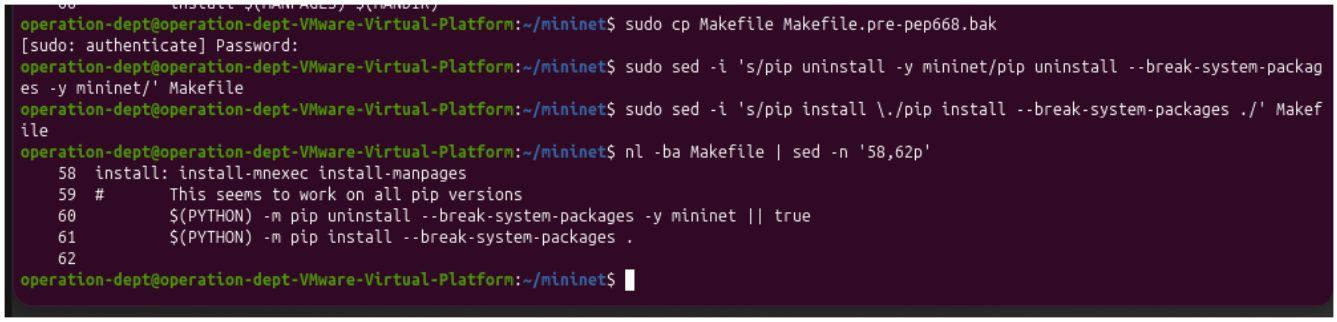
sudo sed -i 's/pip install \./pip install --break-system-
packages ./' Makefile

nl -ba Makefile | sed -n '58,62p'
```

Command	Purpose
cp Makefile …bak	Creates a recovery copy before editing
First sed	Allows the Mininet uninstall check under PEP 668
Second sed	Allows Mininet to install into the system Python environment
nl … sed	Displays the modified lines for verification

This keeps the module's intended Mininet installation process while adapting it for the newer Ubuntu Python policy.

Expected verification



Install Mininet

Install Mininet core files and create the mn command used to build SDN topologies.

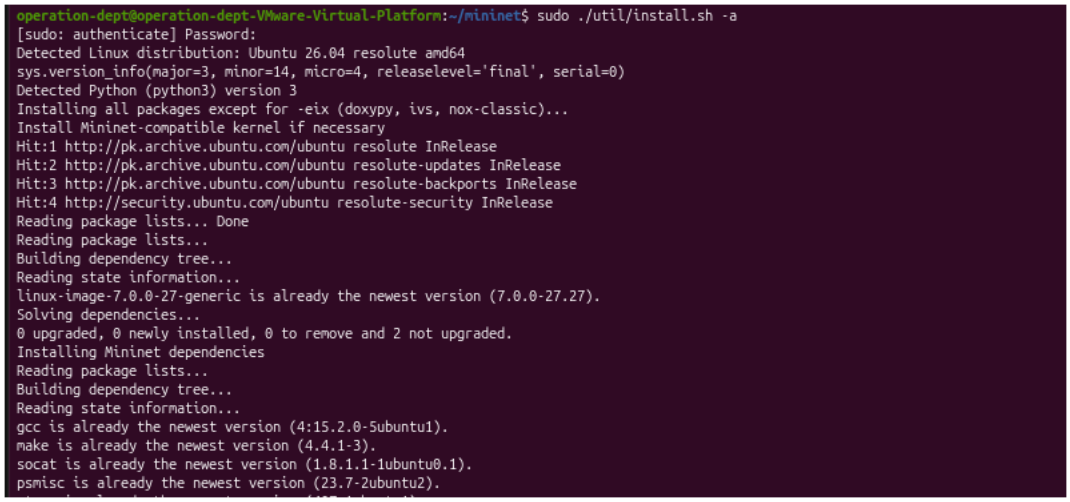
Run Mininet Installation Script

Install:

- Mininet
- OpenFlow support

- Example SDN topologies
- Required Python components

```
sudo ./util/install.sh -a
```



Change OpenFlow Download to HTTPS

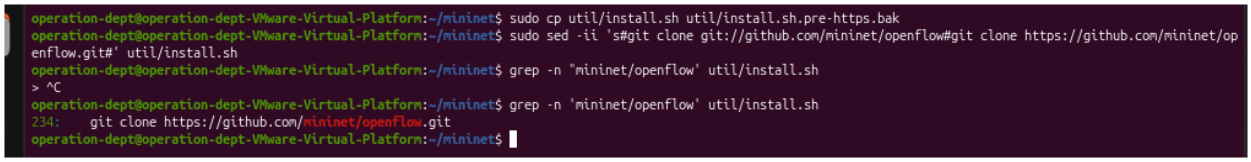
```
sudo cp util/install.sh util/install.sh.pre-https.bak

sudo sed -i 's#git clone git://github.com/mininet/openflow#git
clone https://github.com/mininet/openflow.git#' util/install.sh

grep -n 'mininet/openflow' util/install.sh
```

Command	Purpose
cp	Creates a backup before changing the installer
sed	Replaces the blocked native Git URL with the working HTTPS URL
grep	Confirms that the correct repository address is now present

Expected verification



Test Mininet with Open vSwitch

```
sudo mn --switch ovsbr --test pingall
```

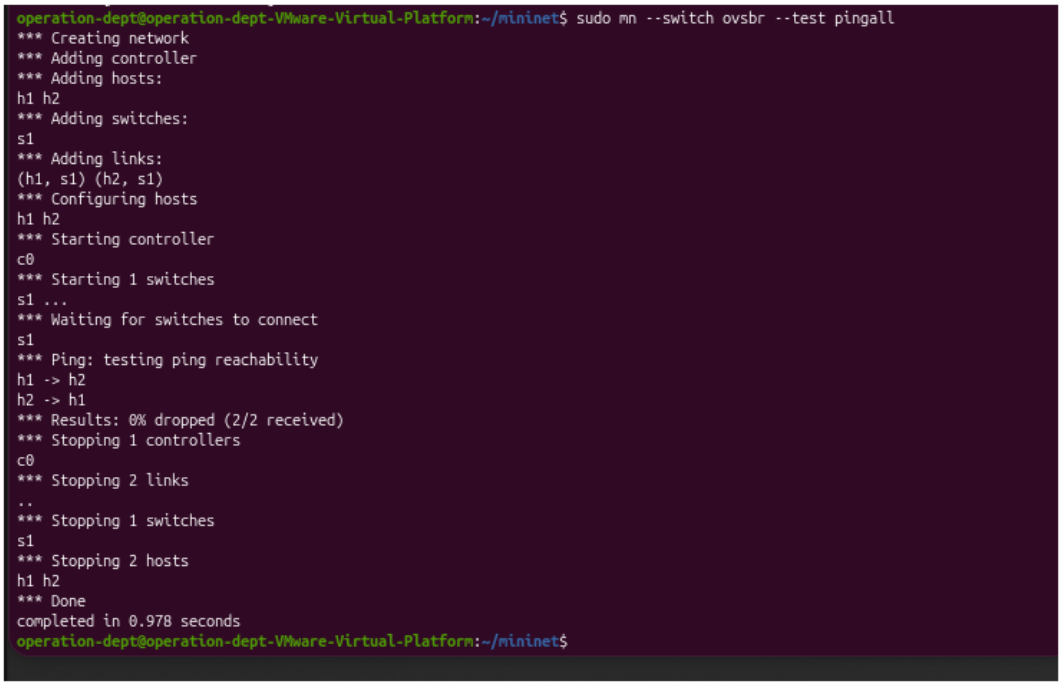
Mininet Installation and Connectivity Test Passed

Your screenshot confirms Mininet is working correctly with Open vSwitch:

*** Results: 0% dropped (2/2 received)

Additional successful indicators:

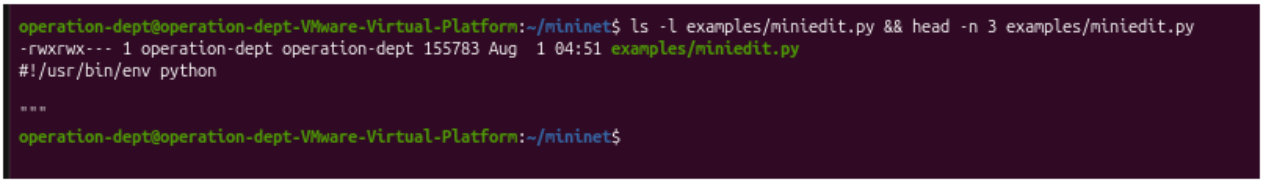
- Hosts h1 and h2 were created.
- Switch s1 started successfully.
- Controller c0 started.
- Both hosts communicated successfully.
- Mininet cleaned up the temporary network correctly.
- Test completed in approximately one second.



Inspect the MiniEdit Script

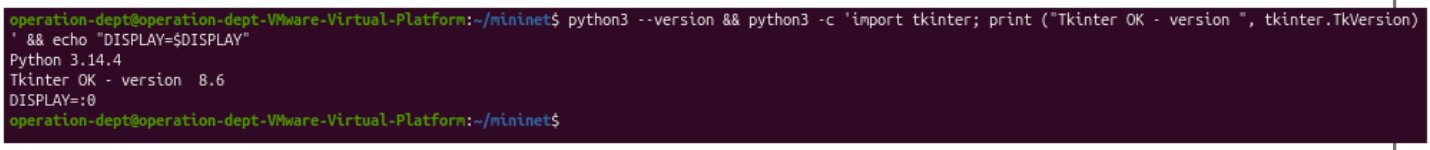
```
ls -l examples/miniedit.py && head -n 3 examples/miniedit.py
```

Part	Purpose
ls -l	Confirms the MiniEdit file exists and shows its permissions
head -n 3	Displays the interpreter line and the first few lines of the script

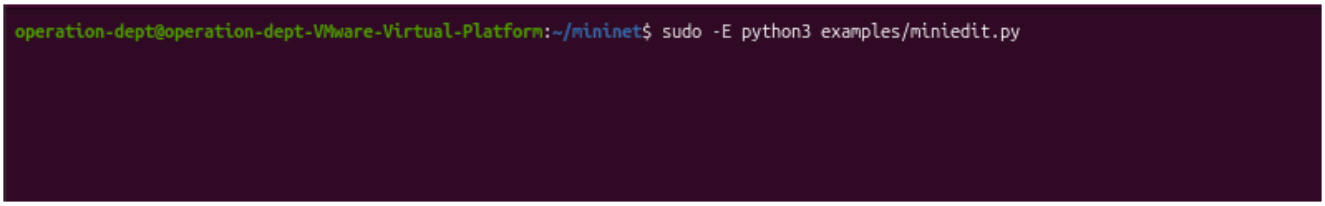


Verify Python 3 and Tkinter

```
python3 --version && python3 -c 'import tkinter; print("Tkinter
OK - version", tkinter.TkVersion)' && echo "DISPLAY=$DISPLAY"
```



Launch MiniEdit

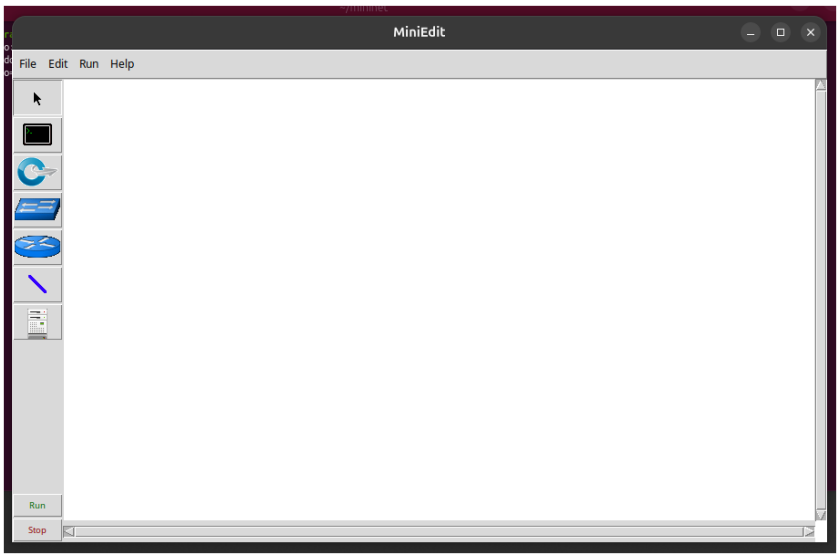


Expected result

A new graphical window titled **MiniEdit** should appear.

You should see:

- A large blank topology canvas
- A toolbar on the left
- Icons for hosts, switches, controllers, and links
- Menus such as **File**, **Edit**, **Run**, and **Help**



Add Two Hosts

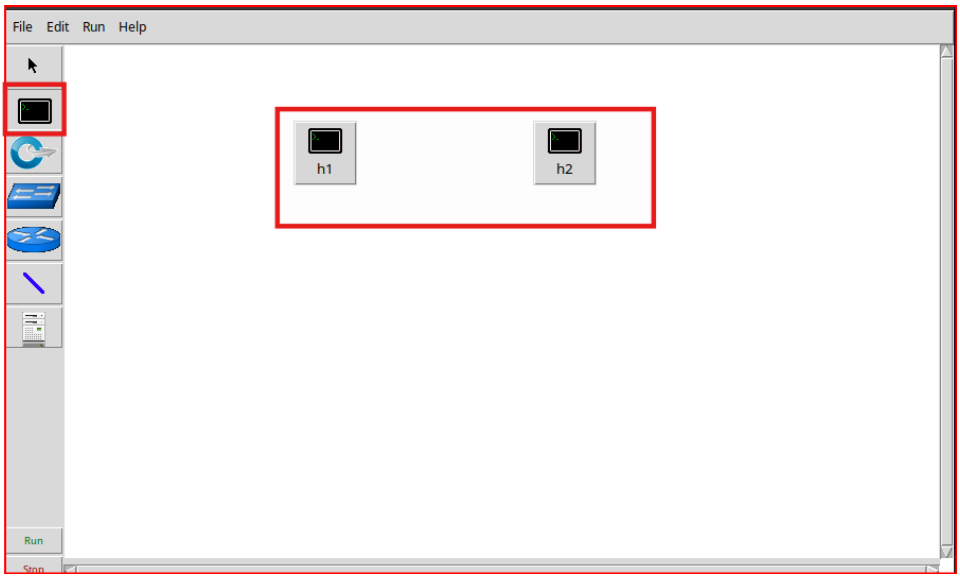
Exact actions

1. On the left toolbar, click the **Host icon**:

Black rectangular terminal icon

It is the **second icon from the top**, directly below the mouse-pointer icon.

2. Click once on the blank canvas in the **lower-left area**.



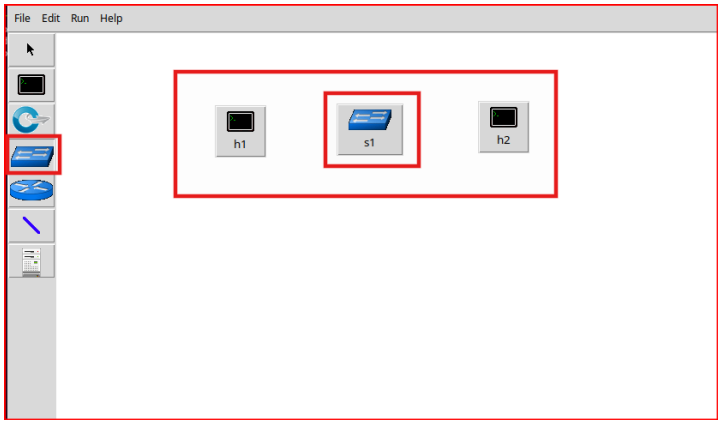
Add the OpenFlow Switch

Steps

1. Click the **Switch icon** on the left toolbar.

It is the **blue rectangular icon**, fourth from the top.

2. Click once between h1 and h2, slightly above them.
3. MiniEdit should create:

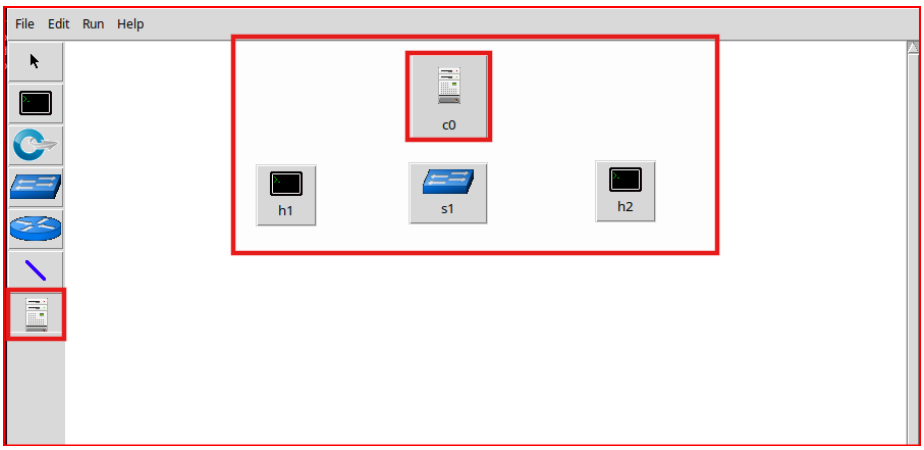


Add the SDN Controller

1. Click the **Controller icon** on the left toolbar.

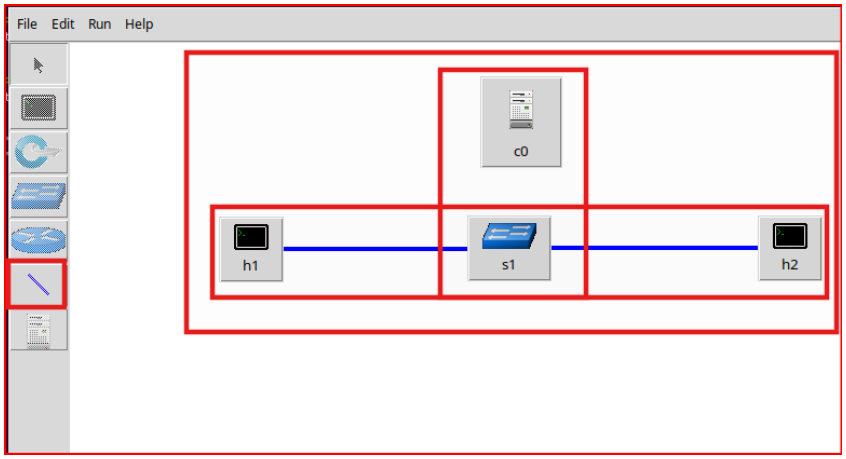
It is the **blue circular icon**, third from the top.

2. Click once above s1.
3. MiniEdit should create:



Connect the Topology

1. Click the Link tool on the left toolbar—the purple diagonal line.
2. Drag from h1 to s1.
3. Drag from h2 to s1.
4. Drag from c0 to s1.



Configure the Controller

Using OVS Controller

The **OpenFlow Reference** controller failed to compile earlier on Ubuntu 26.04. The OVS controller is already available and is suitable for the baseline topology.

Controller Connection Clarification

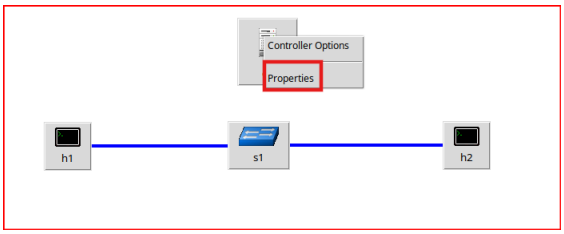
The controller does **not always display a visible cable** to the switch in newer MiniEdit versions. The switch-controller relationship is created logically when the topology starts.

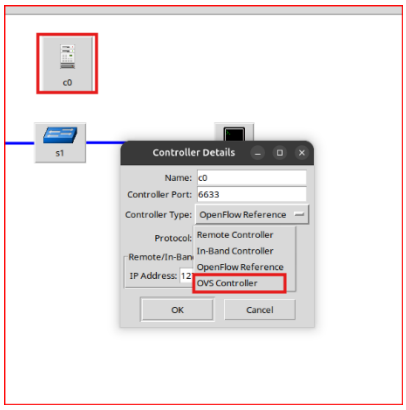
Configure c0

In **Controller Details**:

1. **Controller Type:** OVS Controller
2. **Controller Port:** 6633
3. **Protocol:** keep the default TCP option
4. Click **OK**

Do not select **Remote Controller** because no separate external controller is running.



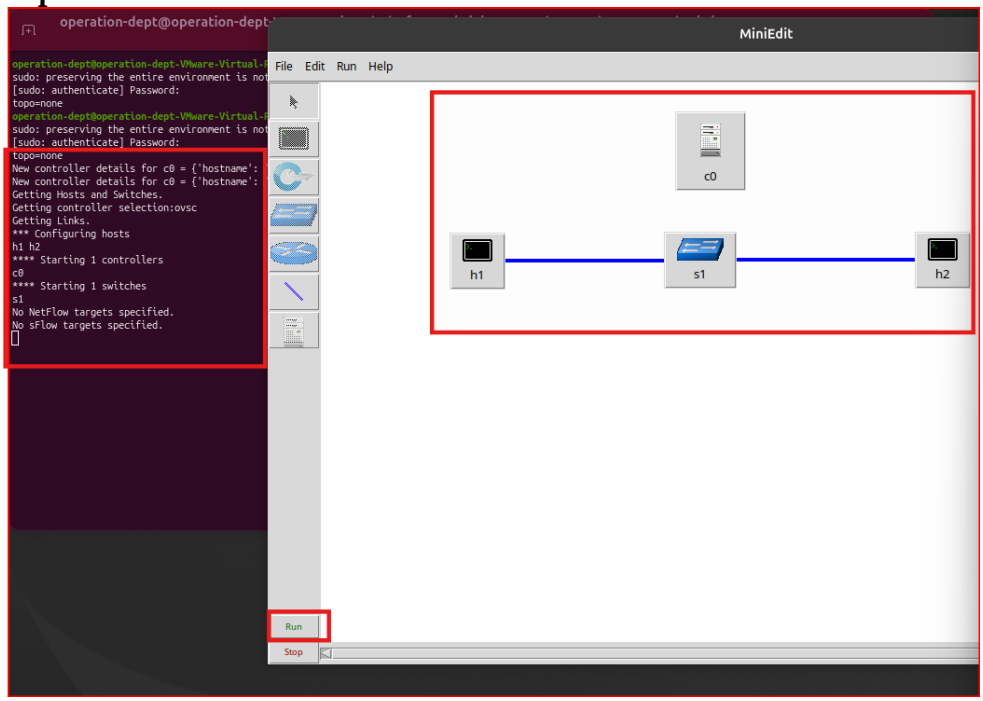


Start the Topology

1. Click the green **Run** button at the bottom-left.
2. Wait approximately 5–10 seconds.
3. Check whether the topology starts without an error.

MiniEdit should automatically associate s1 with c0; the absence of a visible line does not necessarily mean they are disconnected.

Expected result



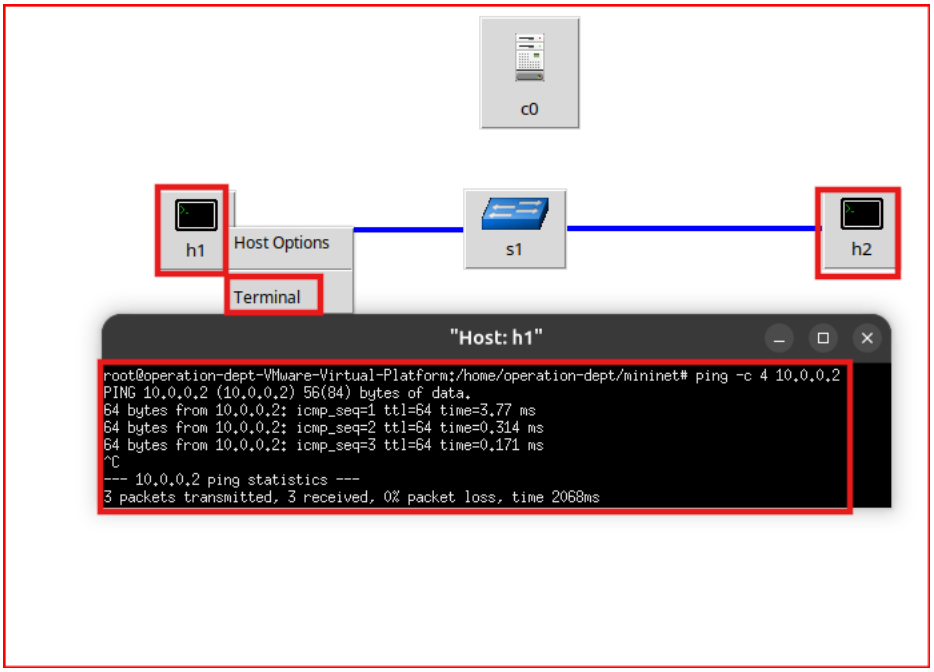
Test Host Connectivity

Keep the topology running.

1. Right-click h1 → **Terminal**.
2. Right-click h2 → **Terminal**.
3. In the h2 terminal, run:

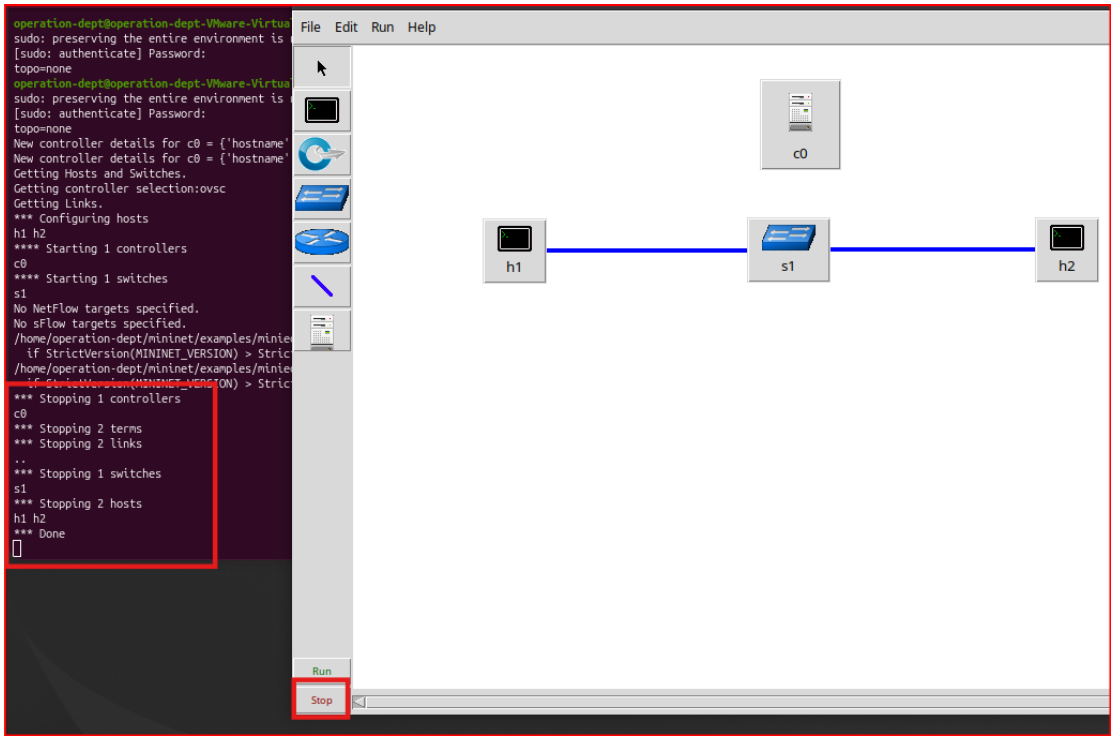
```
ping -c 4 10.0.0.2
```

Expected result



Stop the Topology

1. Close the host terminal.
2. In MiniEdit, click **Stop** at the bottom-left.
3. Wait until the editing icons become active again.



Verify the New Switch

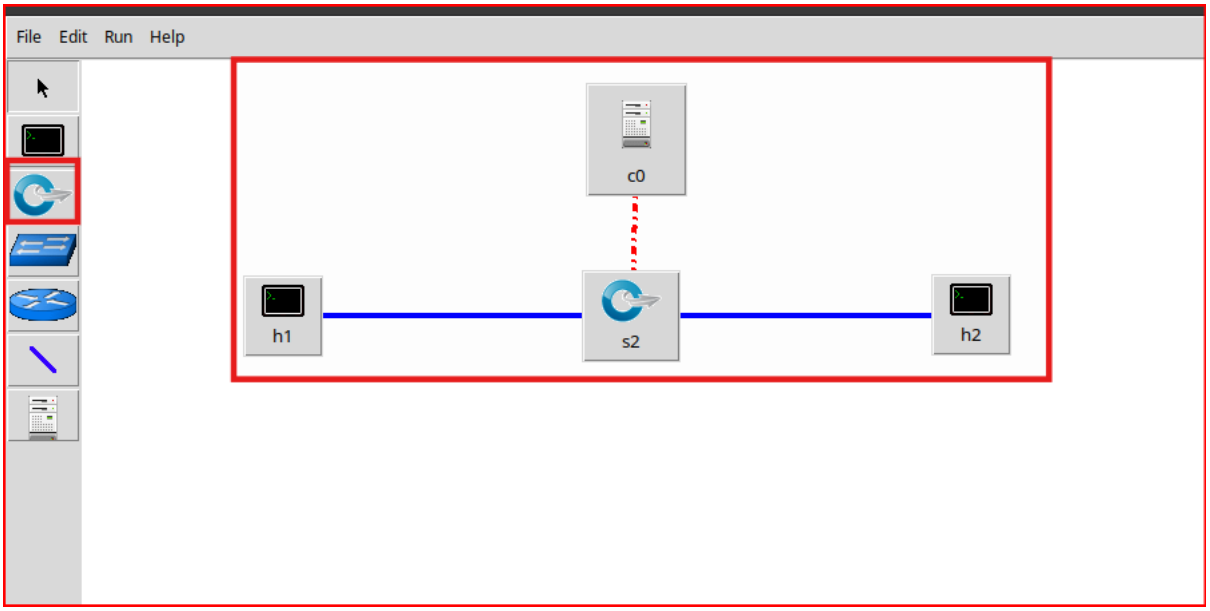
1. Ensure the topology is **stopped**.
2. Keep the new configurable switch in the center.
3. Ensure its name is:

s1

If it appears as s2, rename it to s1 in **Properties**.

4. Reconnect:

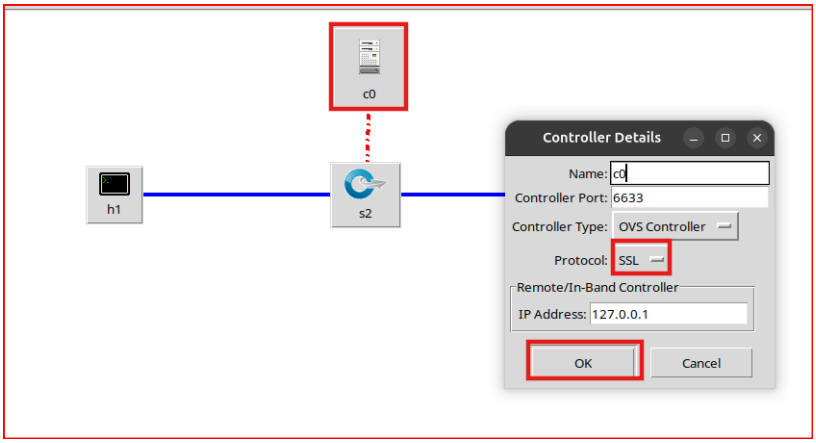
h1 —— s1 —— h2



Set Controller Protocol to SSL

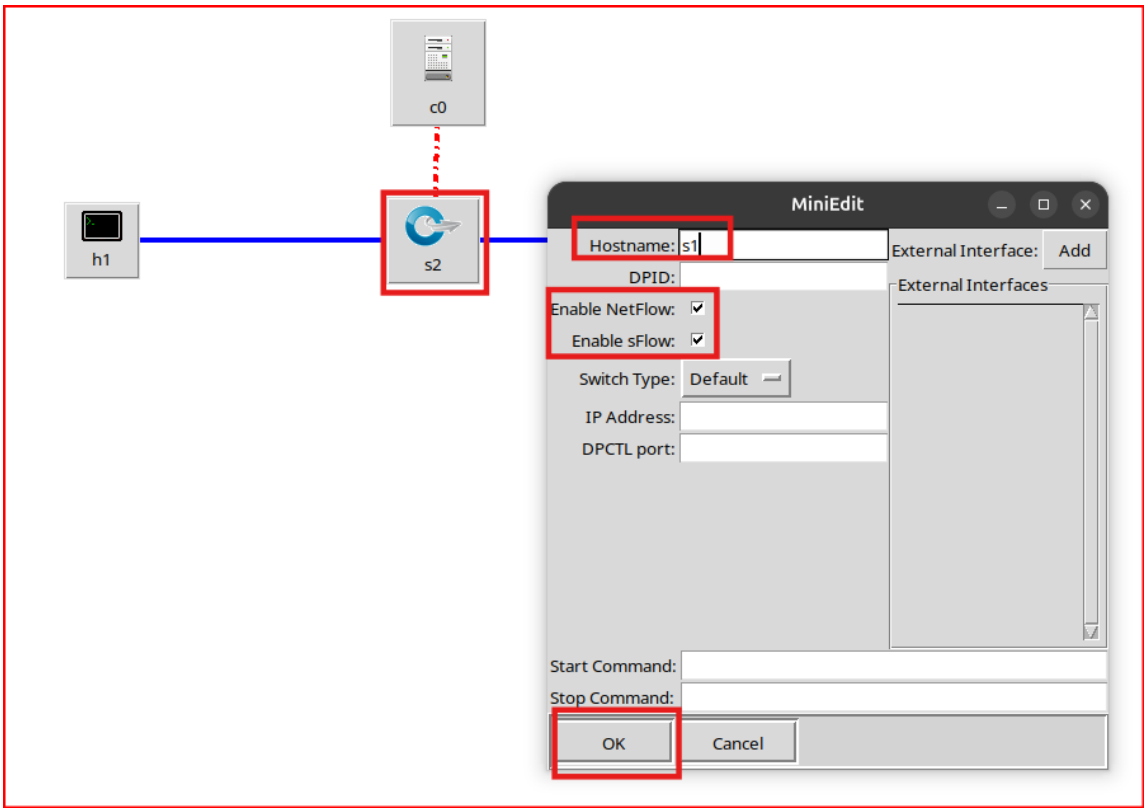
1. Right-click **c0**.
2. Select **Properties**.
3. Keep:
    - **Controller Type:** OVS Controller
    - **Controller Port:** 6633
4. Change **Protocol** from TCP to: SSL

5. Click **OK**.



Enable Monitoring

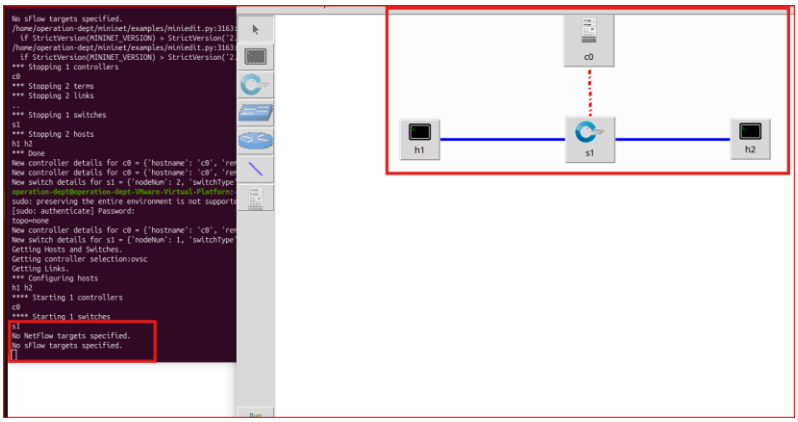
1. Right-click the new s1.
2. Select **Properties**.
3. Open the **MiniEdit** tab.
4. Enable:
    - **NetFlow**
    - **sFlow**
5. Click **OK**.



Start the SSL Topology

1. Click **OK**.
2. Confirm the switch label changes from s2 to s1.
3. Click **Run**.
4. Wait 5–10 seconds.

Warnings about missing NetFlow or sFlow **targets** may appear; these are informational.



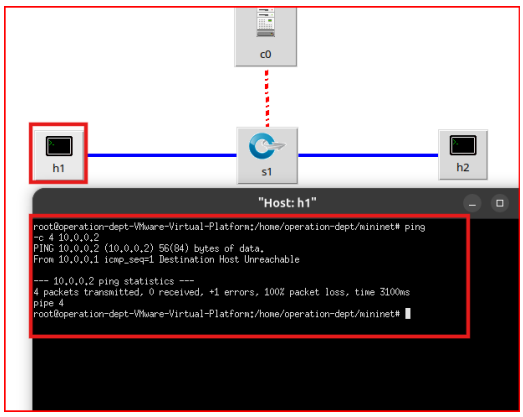
Test Connectivity After SSL

1. Right-click h1.
2. Select **Terminal**.
3. Run:

ping -c 4 10.0.0.2

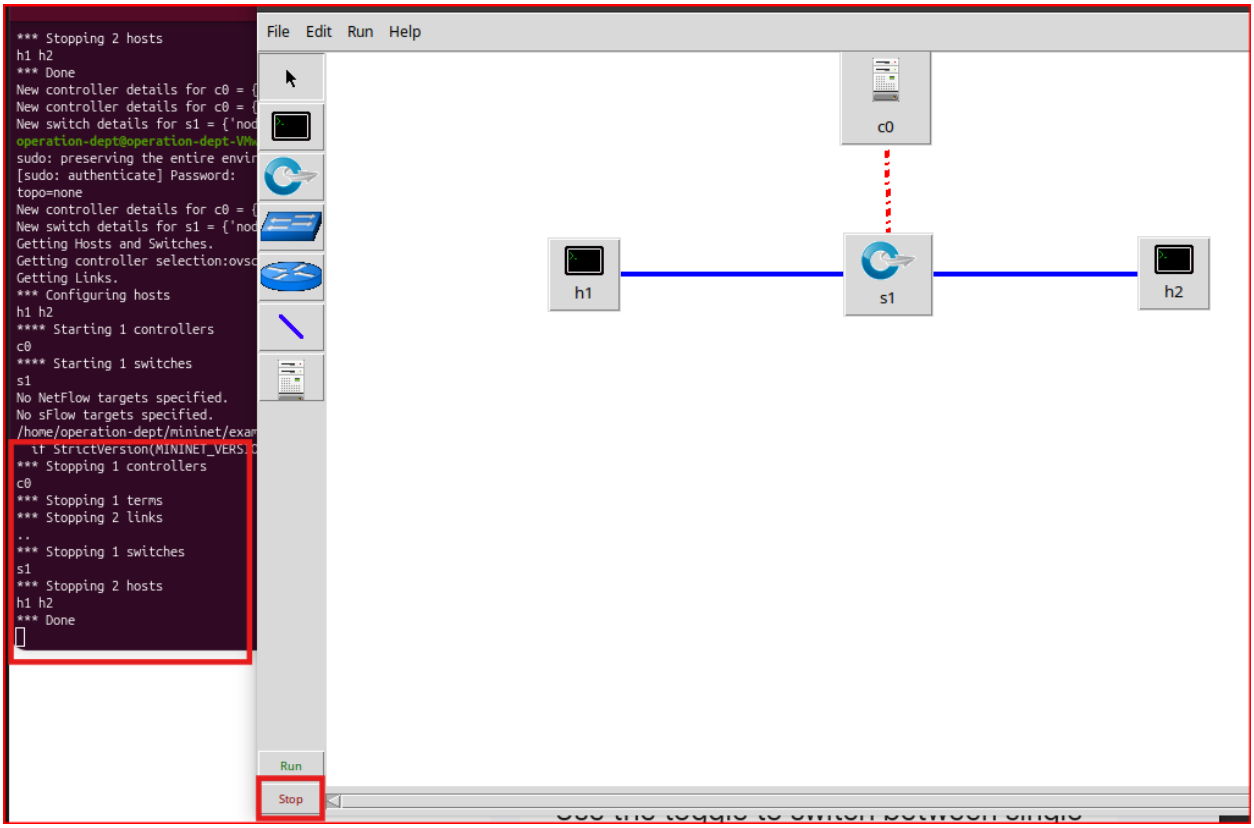
Expected by the module

100% packet loss



Stop the Final Topology

1. Close the h1 terminal.
2. Click **Stop** in MiniEdit.
3. Wait:



Conclusion

Securing SDN Communication Between Switch and SDN Controller Using SSL

In this lab, a virtual Software-Defined Networking environment was successfully created using **Mininet, MiniEdit, Open vSwitch, two virtual hosts, one OpenFlow switch, and one SDN controller**. The topology consisted of hosts h1 and h2 connected through switch s1, while controller c0 provided centralized control of the network.

Before applying SSL, host-to-host communication was tested successfully. Host h1 was able to ping host h2 with **0% packet loss**, confirming that the virtual hosts, links, Open vSwitch, and controller were functioning correctly.

The topology was then stopped, and the controller protocol was changed from the normal controller communication mode to **SSL**. NetFlow and sFlow monitoring options were also enabled on the configurable OpenFlow switch. After restarting the topology, the connectivity test was repeated.

The second ping test produced:

Destination Host Unreachable

100% packet loss

This matched the expected result in the module manual and demonstrated that changing the controller communication configuration affected the switch's ability to receive valid forwarding instructions. Without a functioning controller channel, the switch could not install the necessary flow entries, causing communication between h1 and h2 to fail.

The lab provided practical understanding of:

- SDN network architecture
- Centralized network control
- Mininet host and switch emulation
- Open vSwitch operation
- Controller-switch communication
- SSL protocol selection
- NetFlow and sFlow configuration
- The effect of controller failure on data-plane forwarding
- The importance of securing the SDN control plane